Project Description: Depth Estimation Using Depth Anything

Phase 1: Depth Estimation on a Single Image

- Select an RGB image and load it using OpenCV, Python, or PIL.
- Load the Depth Anything model.
- Generate a depth map for the input image.
- Display the original image and the predicted depth map side by side.
- Save the depth map as an image file.
- Document the model input, output, output dimensions, and the information provided by the depth map.

Phase 2: Numerical Analysis of the Depth Map

- Analyze the depth map using NumPy.
- Calculate its shape, data type, minimum, maximum, mean, and standard deviation.
- Identify the pixel coordinates with the minimum and maximum depth values.
- Calculate the depth value at the center of the image.
- Plot a histogram of depth values.
- Convert the depth map into a NumPy array and save it.
- Explain the technical differences between relative depth and metric depth, and discuss whether a predicted depth value of 0.25 represents an actual distance of 25 cm from the camera.

Phase 3: Model Comparison and Performance Trade-offs

- Select at least two different versions of Depth Anything.
- Run inference on at least 10 images for each model.
- Evaluate and compare the models based on prediction quality, memory usage, FPS, average inference time, and input resolution.
- Measure preprocessing and postprocessing times separately where possible.
- Determine which model is most suitable for a system requiring a minimum of 20 FPS, considering speed, memory consumption, prediction quality, and available hardware.

Phase 4: Video Depth Estimation

- Select a video with a duration of at least 30 seconds.
- Implement a frame-by-frame depth estimation pipeline:

  Input Video → Frame Extraction → Depth Anything → Depth Maps → Output Video

- Generate a depth map for each video frame.
- Combine the predicted depth maps into an output video.
- Calculate the input video's FPS and measure the actual inference FPS.

- Evaluate the system's processing performance.

Phase 5: Depth Visualization and Overlay

- Convert the raw depth map into a colorized depth map using a suitable color map.
- Combine the original RGB image with the colorized depth map to generate a depth overlay.
- Implement an adjustable alpha parameter to control the contribution of the original image and the depth visualization.
- Generate at least five output images using different alpha values.
- Analyze how alpha affects image interpretability and select an appropriate value for presenting depth information to users.

Phase 6: Near and Far Object Analysis

- Select an image containing at least three objects at different distances.
- Identify and define a bounding box or region of interest (ROI) for each object:
    - Object A: Near
    - Object B: Medium distance
    - Object C: Far
- Calculate the mean, median, minimum, and maximum depth values for each ROI.
- Present the results in a comparison table.
- Explain why relative depth values, such as 0.21 and 0.81, cannot automatically be interpreted as physical distances of 2 m and 8 m without a suitable metric depth model and appropriate calibration or validation.

Phase 7: Development of a Practical Depth-Based Warning System

- Design and implement a system that detects objects approaching the camera and generates a warning when an object enters a predefined danger zone.
- Implement the following pipeline:

  Camera → Depth Anything → Depth Map → Nearest Object Estimation → Warning System

- Define a suitable region of interest and a threshold for triggering the warning.
- Process incoming video frames and estimate the proximity of objects within the monitored region.
- Display the following warning when the danger condition is met:

  WARNING!
  OBJECT TOO CLOSE

- Test the system on a video containing objects approaching the camera and evaluate its detection behavior and processing speed.